RICE UNIVERSITY

Framework Fundamentals

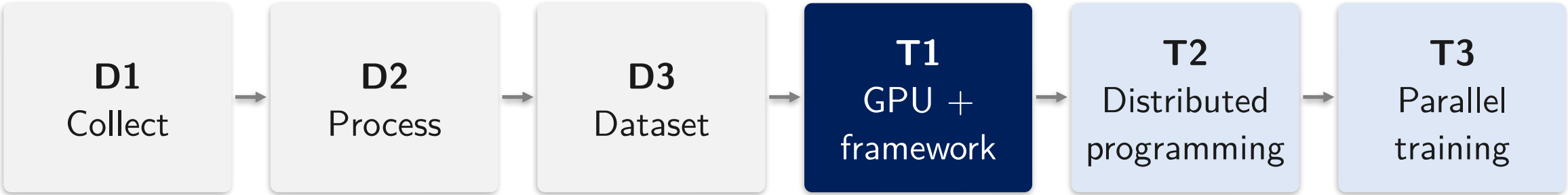
Tensors, dispatch, and the forward pass: what PyTorch does between your Python and the GPU

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 9, Lecture 1

Where we are in the pipeline



Week 8: GPU architecture · Triton kernels

Week 9: the framework layer

In our pipeline

Today: what PyTorch does **between your Python and the GPU** — tensors, the forward pass, dispatch, and the memory of one training step.

Lecture 2 this week: graphs and compilation (torch.compile, torch.func) — the other half of T1.

Why do we need a framework?

Training our MLP with only last week's skill — one Triton kernel at a time. What would you write by hand?

Kernels
$\approx$ **25** per step:
forward, backward, update

Gradients
the derivative of every op,
for all **5** parameter tensors

Memory
allocate, size, keep alive,
and free every buffer

... and all of it again every time the model changes.

Key idea

With PyTorch the whole step is a few lines: `loss = F.cross_entropy(model(x), y)`, `loss.backward()`, `opt.step()`.

Source: Operation counts from a TorchDispatchMode trace of the course MLP (Part 3).

What a framework is, in plain words

Your model code (Python): `model(x)`, `loss.backward()`, `opt.step()`

The framework (PyTorch)

Tensors	**Operators**	**Dispatcher**	**Autograd**	**Allocator**
arrays + layout	matmul, tanh, ...	picks the kernel	gradients (provided)	GPU memory

Kernels: cuBLAS, PyTorch's CUDA kernels, Triton

GPU hardware (Week 8)

Key idea

A **framework** turns a few lines of Python into the right sequence of GPU kernels.

Where frameworks show up

- **PyTorch** is this course's framework, and that of Llama's reference code and Hugging Face.
- **T2, T3:** torch.distributed and DDP are built on the same tensors and kernels.
- **T1, Lab 3:** trace one step; compare eager with compiled execution.

The price

Python + dispatch + launch for **each** of the **59** calls in one step

Small models: the overhead dominates (Lecture 2: compile)

In our pipeline

Why a framework? You write the math once; it supplies the kernels, gradients, and memory management — at a per-op cost we will learn to measure.

Source: Operation count from a TorchDispatchMode trace of the course MLP (Part 3).

Learning goals for today

- Describe a tensor as **(storage, offset, shape, strides, dtype, device)**; tell a **view** from a **copy**.
- Connect **strides** to last week's **coalescing**.
- Break a **forward pass** into ATen ops; estimate each op's **FLOPs, bytes, and intensity**.
- Trace a call from Python through the **dispatcher** to a **kernel**; explain eager per-op overhead.
- Account for **parameters, gradients, optimizer state, and activations**, and predict peak memory.

A puzzle to start

Our MLP
34,338 parameters
FP32 $\rightarrow$ 137.4 KB

One training step
batch 64, Adam optimizer
forward, backward, update

At its peak, how much GPU memory does one training step hold?

Key idea

$\approx$ **0.71 MB** — about **5×** the parameters. At batch 4096: $\approx$ **20 MB**, $\approx 140\times$. Today we find every byte.

Part 1

Tensors: storage and layout

What exactly is a tensor in memory?

A tensor is metadata over storage

storage
which block of memory

offset
first element: 0

shape
(3, 4)

strides
(4, 1): elements per step

dtype
float32: 4 bytes each

device
cpu, cuda:0, ...

Key idea

A tensor is a **view** of storage. Many tensors can share one storage.



storage: 12 float32 values, 48 bytes, on one device

Source: PyTorch documentation: torch.Tensor, Tensor Views (pytorch.org/docs).

From index to address

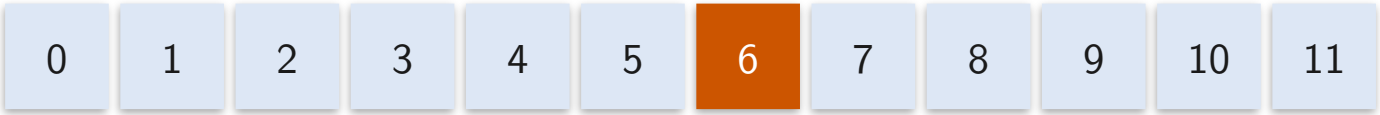
$$\text{position}(i, j) = \text{offset} + i \cdot s_0 + j \cdot s_1$$

$$\text{byte address} = \text{base} + \text{position} \times \text{itemsize}$$

a: shape $(3, 4)$, strides $(4, 1)$, offset 0

$$a[1, 2] \to 0 + 1 \cdot 4 + 2 \cdot 1 = 6$$

$$\text{byte } 6 \times 4 = 24$$



storage of a: position 6 holds a[1, 2]

Key idea

General rule: position = offset + Σ index $\times$ stride — a **linear** map from indices to memory.

Views share storage: transpose

a (3 × 4), strides (4, 1)

0	1	2	3
4	5	6	7
8	9	10	11

a.T (4 × 3), strides (1, 4)

0	4	8
1	5	9
2	6	10
3	7	11

cell numbers = storage positions

$$a.T[2,\ 1]$$
$$\rightarrow 2 \cdot 1 + 1 \cdot 4 = 6$$
$$= a[1,\ 2]$$

- Same storage, same offset
- Only shape and strides change
- **O(1)**: no bytes move

Key idea

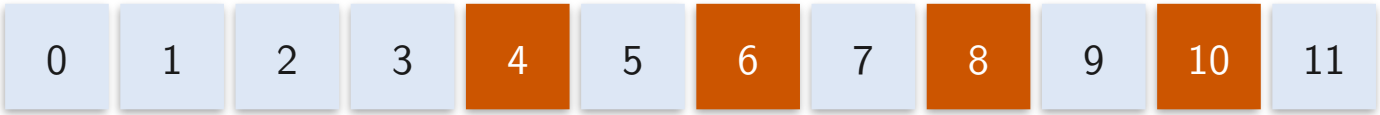
`a.T` is a **view**: new metadata, same bytes. It is free — until a kernel reads it.

Slicing is a view too

```
a = torch.arange(12.).reshape(3, 4)
s = a[1:, ::2]
s.shape              # (2, 2)
s.stride()           # (4, 2)
s.storage_offset() # 4
```

$$s[1, 1] \to 4 + 1 \cdot 4 + 1 \cdot 2 = 10$$

- Views **alias**: writing to s changes a.
- Index with a tensor or mask $\to$ a **copy**.



s reads storage positions 4, 6, 8, 10

Source: PyTorch documentation: torch.Tensor, Tensor Views (pytorch.org/docs).

Contiguous or not?

$$\text{contiguous} \iff \text{strides} = \text{row-major strides of the shape}$$

Operation	Returns	Bytes moved
`a.T`, `a[1:, ::2]`, `a.view(...)`	view	0
`a.T.contiguous()`	copy (new storage)	read + write all
`a.T.view(12)`	**error**: strides incompatible	—
`a.T.reshape(12)`	copy, silently	read + write all

Watch out

`.is_contiguous()` tells you which case you are in. A hidden copy costs a full read and write.

Strides meet coalescing

```
@triton.jit
def scale(x, y, n, sr, sc, BLOCK: tl.constexpr):
    row = tl.program_id(0)
    col = tl.arange(0, BLOCK)
    m = col < n
    v = tl.load(x + row*sr + col*sc, mask=m)
    tl.store(y + row*n + col, 2*v, mask=m)

scale[(M,)](x, y, N, x.stride(0), x.stride(1),
            BLOCK=1024)    # assumes N <= 1024
```

Input	sc	Used
x	1	100%
x.T	4096	12.5%

x: 4096 × 4096, FP32

Memory

The **last-dimension stride** decides coalescing. Pass strides into every kernel you write.

dtypes and bytes

dtype	Bytes	4096^2 matrix	MLP params	Typical use
`float64`	8	134.2 MB	274.7 KB	checks
`float32`	4	67.1 MB	137.4 KB	reference, Adam state
`bfloat16` / `float16`	2	33.6 MB	68.7 KB	training default
`int64`	8	134.2 MB	—	token ids (default)
`int8` / `bool`	1	16.8 MB	—	quantized, masks

$$\text{bytes} = \text{numel} \times \text{itemsize}$$

Key idea

In code: `x.nbytes` gives it directly.

Source: PyTorch documentation, torch.Tensor.element_size; sizes computed.

Check yourself

Two minutes with a neighbour.

- `x = torch.empty(2, 3, 4)`. What are the shape and strides of `x.permute(2, 0, 1)`?
- `u = torch.randn(4, 4)`. Shape, stride and offset of `u[:, 1]`? Is it contiguous?
- You pass `x.T` (2048 $\times$ 2048, FP32) to a Triton kernel that assumes stride 1. What goes wrong?

Part 2

The forward pass, op by op

What does PyTorch actually run when you call model(x)?

From model code to ATen ops

```
class MLP(nn.Module):
    # emb, fc1, fc2 are created in __init__
    def forward(self, ids):              # (B, 8)
        e = self.emb(ids)                # (B, 8, 16)
        x = e.view(len(ids), -1)         # (B, 128)
        h = torch.tanh(self.fc1(x))      # (B, 64)
        return self.fc2(h)               # (B, 322)

logits = model(ids)
loss = F.cross_entropy(logits, y)
```

aten::embedding

aten::view

aten::addmm

aten::tanh

aten::addmm

aten::log_softmax

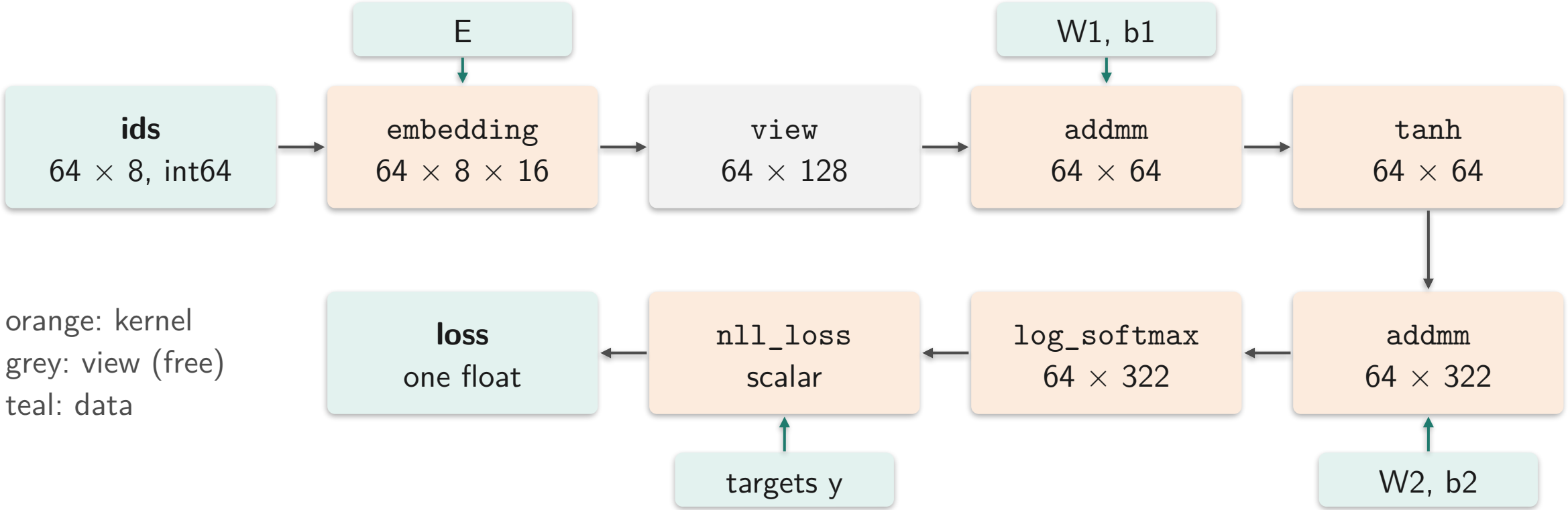
aten::nll_loss

Key idea

One line of Python can be **several** ATen ops; `cross_entropy` is two.

Source: PyTorch documentation: torch.nn.Linear, torch.nn.functional.cross_entropy, TorchDispatchMode.

The forward data flow, B = 64



Key idea

6 kernels do the work; `view` (and `nn.Linear`'s weight transposes) cost no bytes.

Counting one op: the output layer

$$\text{addmm:}\quad (64 \times 64) \cdot (64 \times 322) \;+\; \text{bias}\ (322)$$

$$F = 2 \cdot 64 \cdot 64 \cdot 322 + 64 \cdot 322 = 2{,}658{,}432\ \text{FLOPs}$$

$$Q = 4 \cdot (64{\cdot}64 + 64{\cdot}322 + 322 + 64{\cdot}322) = 182{,}536\ \text{bytes}$$

$$I = F\ /\ Q \approx 14.6\ \text{FLOP/byte}$$

Memory

FP32 ridge $\approx$ **20 FLOP/byte**: even our biggest op is **memory-bound** on paper.

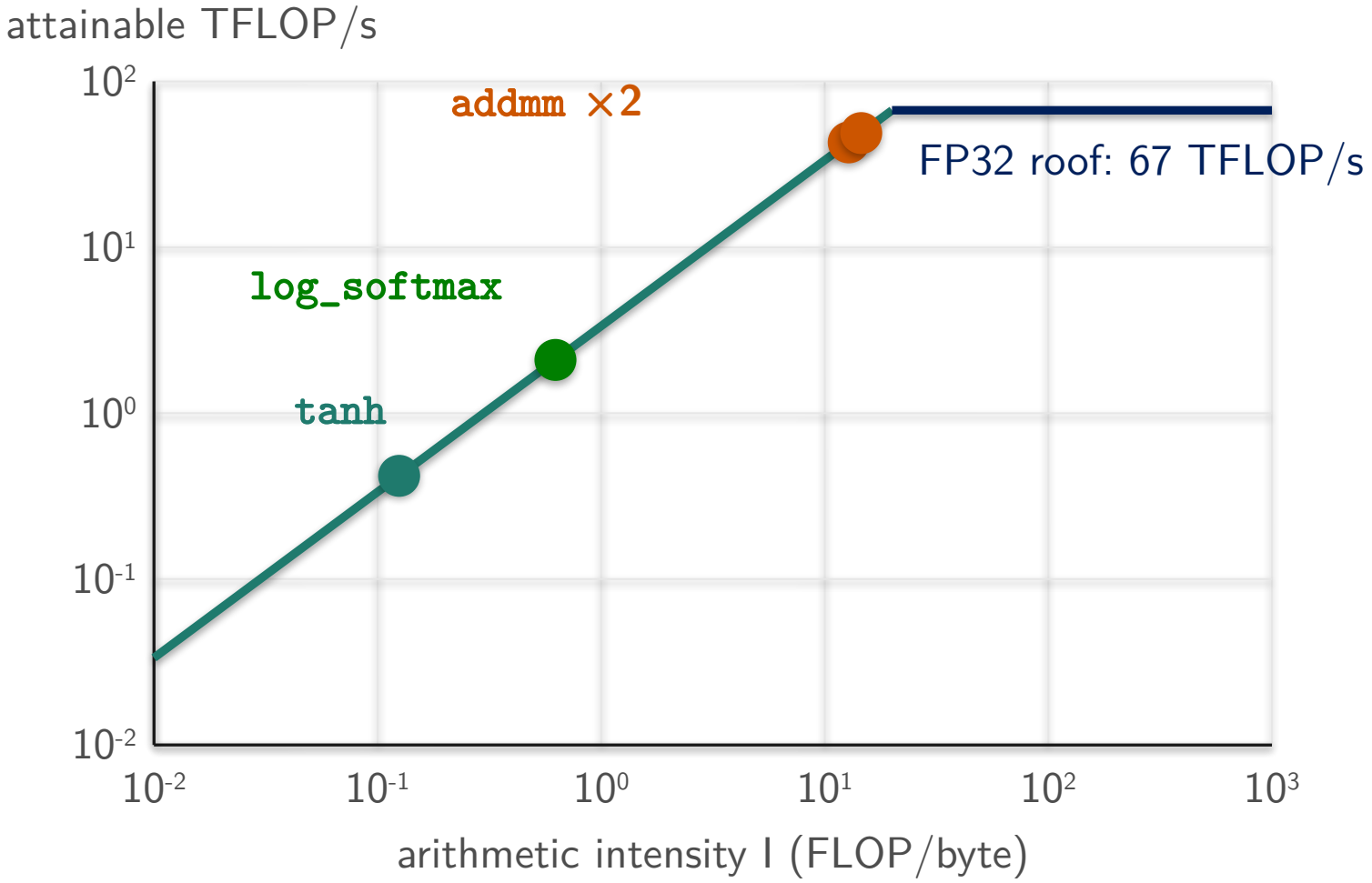
Source: H100 datasheet (SXM5): ridge = 67 TFLOP/s ÷ 3.35 TB/s; FLOPs and bytes computed.

Every forward op: FLOPs, bytes, intensity

ATen op	Output	FLOPs	Bytes	FLOP/byte	Kind
`embedding`	$64 \times 8 \times 16$	0	69.6 KB	0	kernel
`view`	64×128	0	0	—	**view**
`addmm`	64×64	1.05 M	82.2 KB	12.81	kernel
`tanh`	64×64	4,096	32.8 KB	0.12	kernel
`addmm`	64×322	2.66 M	182.5 KB	14.56	kernel
`log_softmax`	64×322	103 K	164.9 KB	0.62	kernel
`nll_loss`	scalar	64	0.8 KB	0.08	kernel
Forward		**3.82 M**	**532.7 KB**	7.2	6 kernels

Source: Computed for the course MLP at B = 64, FP32; FLOP-per-element counts for tanh and log_softmax are estimates.

The forward ops on the roofline



- All left of the **ridge** ≈ 20
- Matmuls $\approx$ 13–15; elementwise < 1
- `embedding`: $\mathrm{I} = 0$, a pure gather

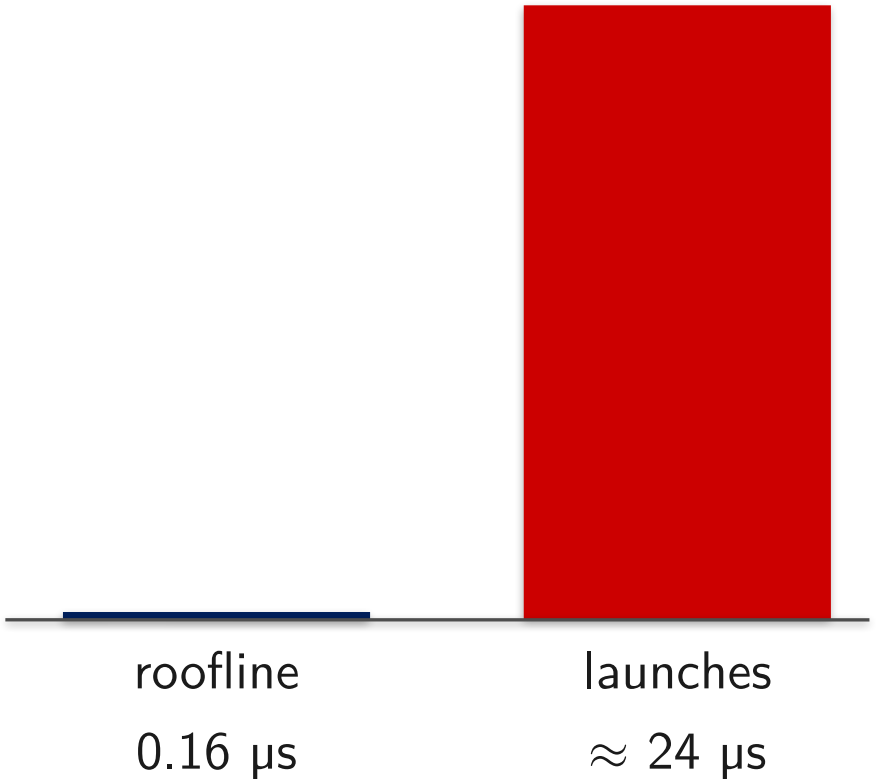
Source: Williams, Waterman & Patterson, "Roofline," Communications of the ACM 52(4), 2009; H100 datasheet (SXM5).

Where the forward time goes

- Roofline time, all 6 kernels: $\approx$ **0.16 µs**
- 6 launches $\times$ $\approx$ 4 µs $\approx$ **24 µs**
- $\approx$ **151×** the roofline: **launch-bound**

In our pipeline

The forward pass of a small model is **launch-bound**.
Lecture 2: fewer, bigger kernels.



Source: Roofline times computed from common.SPEC; $\approx$ 4 µs per launch is the Week 8 round assumption.

How autograd works, at a high level

1. Forward: run each operation and record it



2. Every operation comes with its own derivative rule (built into PyTorch)

3. Backward: walk the record in reverse, applying each rule (the chain rule)

Key idea

When the walk reaches a parameter, its gradient lands in **p.grad**. You never write a backward; you only need this picture.

Source: PyTorch documentation, "Autograd mechanics".

Autograd as a provided tool



Gives

`.grad` for all **34,338** parameters: 137,352 bytes

Costs

$\approx$ **2×** forward FLOPs: $\approx$ 7.4 MFLOP at B = 64

Keeps

forward **activations** alive until backward: 131,584 bytes

In this course

You write the **forward pass**; the framework computes every gradient. Course code never writes a backward.

Source: PyTorch documentation, torch.Tensor.backward; Kaplan et al. 2020 (forward 2N, backward 4N FLOPs per token).

Forward-only runs: no_grad and inference_mode

Mode	Keeps activations?	After forward, B = 64	Use for
default	yes	+131.6 KB	training
`torch.no_grad()`	no	$\approx$ +0 (just the loss)	evaluation
`torch.inference_mode()`	no, and leaner	$\approx$ +0	inference

```
model.eval()
with torch.inference_mode():
    val = F.cross_entropy(model(xv), yv)
```

Watch out

Evaluating without `no_grad` keeps activations alive: a common memory leak.

Source: PyTorch documentation: `torch.no_grad`, `torch.inference_mode`, `torch.nn.Module.eval`.

Check yourself

Two minutes with a neighbour.

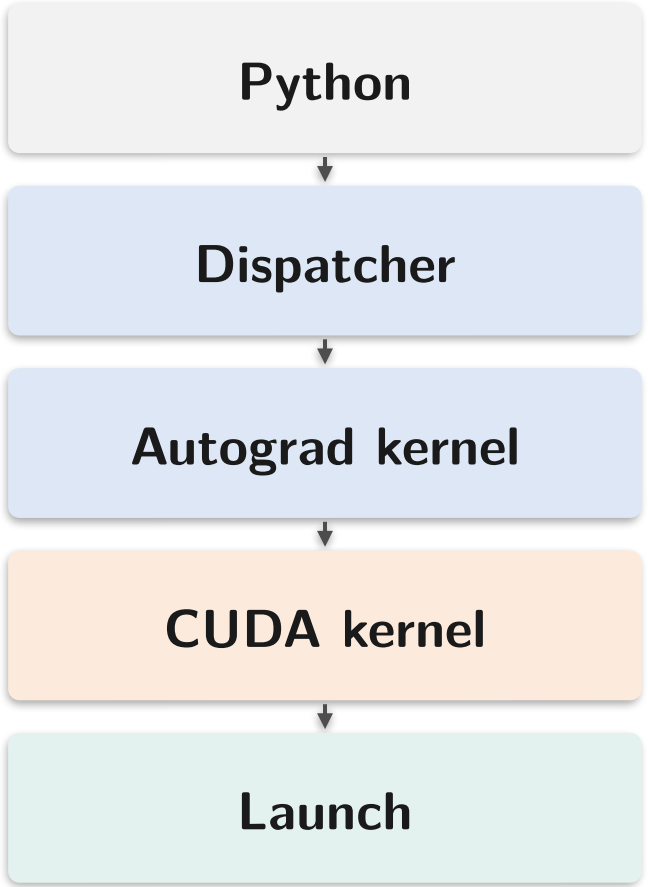
- Batch 128: FLOPs, bytes and intensity of the output `addmm`? Above the ridge yet?
- Is the forward pass still launch-bound at $B = 4096$? (6 kernels, ≈ 4 µs per launch)
- `total += loss` in an evaluation loop, without `no_grad`. What happens to memory?

Part 3

Operator dispatch

What happens between a Python call and a GPU kernel?

The journey of one call



`out = x @ W`: interpreter, argument parsing

key set from inputs + thread-local state

bookkeeping for backward, then redispatch

choose float32 implementation (cuBLAS)

enqueue on a stream, return immediately

CPU, every op

GPU work starts later

Source: PyTorch documentation, "Extending PyTorch: the dispatcher"; E. Yang, "Let's talk about the PyTorch dispatcher" (2020).

Dispatch keys: one call, several layers

```
t = torch.randn(2, 3)
torch._C._dispatch_keys(t)
# DispatchKeySet(CPU, ADInplaceOrView,
#                AutogradCPU, AutocastCPU)
```

Autocast — cast dtypes, if enabled

Autograd — backward bookkeeping

ADInplaceOrView — view and version bookkeeping

CPU / CUDA — the actual kernel

priority: high → low

Key idea

New features — autograd, autocast, vmap, compile tracing, new devices — are new **keys**, not edits to every kernel.

Source: PyTorch documentation, "Extending PyTorch: the dispatcher"; E. Yang, "Let's talk about the PyTorch dispatcher" (2020).

Launches are asynchronous

Large kernels: CPU runs ahead, GPU stays busy



Tiny kernels: GPU waits for the CPU



Watch out

With tiny kernels, step time = **CPU cost per op** × **number of ops**: overhead-bound.

Counting the operations in one step

Phase	ATen calls	Examples
Forward	11	`embedding`, `view`, `addmm` $\times 2$, `tanh`, `log_softmax`, …
Backward	29	`mm` $\times 4$, `tanh_backward`, `embedding_dense_backward`, …
Adam (foreach)	19	`_foreach_lerp_`, `_foreach_sqrt`, `_foreach_addcdiv_`, …
One step	**59**	≈ 25 of them launch GPU kernels

$$\approx 25 \text{ launches} \times \approx 4\ \mu\text{s} \approx 100\ \mu\text{s} \quad \text{vs} \quad \approx 0.17\ \mu\text{s of arithmetic}$$

In our pipeline

Eager mode pays Python + dispatch + launch **per op**. Lecture 2: capture many ops as **one graph**.

Source: Counts from a TorchDispatchMode trace of the course MLP; per-launch cost is an assumed round number.

Check yourself

- When you call `x @ W`, what decides which kernel runs?
- Does wrapping a tiny-op loop in `torch.no_grad()` remove its per-op overhead? Why or why not?
- You time `y = x @ W` without `torch.cuda.synchronize()`. What did you measure?

Part 4

The memory of a training step

Where does every byte go?

Four kinds of memory

Kind	Size	Created	Freed
Parameters θ	P	model init	never
Gradients $\partial L/\partial\theta$	P	backward	`zero_grad()` (set to None)
Adam state m, v	2P	first `opt.step()`	never
Activations	$\propto$ batch	forward	during backward

Memory

Three kinds scale with the **model** (P, P, 2P); activations scale with the **batch**.

Source: Kingma & Ba, "Adam," ICLR 2015; torch.optim.Adam documentation.

Parameters and gradients, exactly

Tensor	Shape	Count
Embedding E	322×16	5,152
W1, b1	128×64, 64	8,256
W2, b2	64×322, 322	20,930
Total P		**34,338**

$$\theta\text{: } 34{,}338 \times 4$$

$$= 137{,}352 \text{ bytes}$$

$$\text{gradients: } 137{,}352 \text{ bytes}$$

Key idea

The output layer holds **61%** of the parameters: the vocabulary is the widest dimension.

Adam keeps two more copies

$$m \leftarrow \beta_1\, m + (1 - \beta_1)\, g$$

$$v \leftarrow \beta_2\, v + (1 - \beta_2)\, g \odot g$$

$$\theta \leftarrow \theta - \eta \cdot m \;/\; (\mathrm{sqrt}(v) + \varepsilon)$$

Optimizer	Extra state
SGD	0
SGD + momentum	P
Adam / AdamW	**2P**

Memory

θ + gradient + m + v = **16 bytes per parameter** (FP32) $\rightarrow$ 549,408 bytes for our MLP.

Source: Kingma & Ba, "Adam," ICLR 2015; torch.optim.Adam documentation.

Activations kept for backward

Kept tensor	Shape	Bytes, B = 64
embedded input x	B × 128	32,768
hidden h	B × 64	16,384
log-probabilities	B × 322	82,432
Total	B × 514	**131,584**

$$A(B) = 4 \cdot 514 \cdot B$$
$$= 2{,}056 \cdot B \text{ bytes}$$

linear in the batch

Key idea

The log-probs are **63%** of the activations: again, the vocabulary.

Memory over one training step



- $\theta + m + v \approx 412$ KB
- Forward adds **activations**
- Peak adds two $\mathbf{B} \times \mathbf{V}$ gradients
- Gradients arrive **after** the peak

Predicting the peak

$$\text{peak} \approx 3P \cdot 4 \;+\; \text{batch} \;+\; A(B) \;+\; 2 \cdot 4 \cdot B \cdot V$$

$$= 412{,}056 + 4{,}608 + 131{,}584 + 164{,}864$$

$$= 713{,}112 \text{ bytes} \approx 0.71 \text{ MB}$$

Check

Prediction vs. a measured run: within **1%** at B = 64. You will check this on the GPU.

Scaling the batch

Batch B	$\theta + m + v$	A(B)	B × V grads	Peak
64	0.41 MB	0.13 MB	0.16 MB	**0.71 MB**
256	0.41 MB	0.53 MB	0.66 MB	**1.62 MB**
1,024	0.41 MB	2.11 MB	2.64 MB	**5.23 MB**
4,096	0.41 MB	8.42 MB	10.55 MB	**19.68 MB**

Memory

Activations overtake the static memory near $\mathbf{B \approx 200}$. For small models, the **batch** sets the memory.

Scaling the model: 16 bytes per parameter

Model	Parameters	16 bytes × P	Fits on one 80 GB GPU?
Course MLP	34,338	0.55 MB	easily
GPT-2 small	124 M	2.0 GB	yes
1.3 B model	1.3 B	20.8 GB	yes, with room for activations
7 B model	7 B	**112 GB**	**no** — before any activation

$$\text{FP32 Adam: } 4 + 4 + 4 + 4 = 16$$

$$\text{mixed precision: } 2 + 2 + 4 + 4 + 4 = 16$$

In our pipeline

Week 12: **shard** the 16 bytes across GPUs.

Source: Rajbhandari et al., "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. GPT-2: Radford et al. 2019; H100: 80 GB (datasheet).

Check yourself

- Replace Adam with plain SGD. How much memory does one step save for our MLP?
- Double the hidden width to 128. Which activations grow, and what is A(64) now?
- How many 80 GB GPUs does a 13 B-parameter model need for parameters, gradients, and Adam state?

IN-CLASS BUILD

Trace one training step

Work in pairs. Predict first; measure in Lab 3.

Exercise: predict, then measure

Quantity (steady state)	B = 64	B = 1024	Measured
1. Memory at the start of a step			
2. Increase after the forward pass			
3. Peak during the step			
4. Memory after `zero_grad()`			

USE

$P = 34{,}338$, $V = 322$, $A(B) = 2{,}056 \cdot B$

ids and targets are int64

STRETCH

Which rows differ in **step 1**, and why?

How to measure it

```
for _ in range(2): step(x, y)        # warm up
torch.cuda.synchronize()
base = torch.cuda.memory_allocated()
torch.cuda.reset_peak_memory_stats()
loss = F.cross_entropy(model(x), y)
fwd = torch.cuda.memory_allocated() - base
loss.backward(); opt.step(); opt.zero_grad()
torch.cuda.synchronize()
peak = torch.cuda.max_memory_allocated() - base
```

```
from torch.profiler import profile
with profile(profile_memory=True) as prof:
    step(x, y)
print(prof.key_averages().table(
    sort_by="self_cuda_memory_usage"))
```

- **Warm up**: Adam state appears at step 1.
- **Subtract** a baseline.
- **Synchronize** before reading.
- Allocator rounds to **512 bytes**.

Source: PyTorch documentation: torch.cuda memory management; torch.profiler.

Exercise: solutions

Quantity	B = 64	B = 1024
1. Start: 12P bytes + batch	416.7 KB	485.8 KB
2. Forward adds A(B)	131.6 KB	2.11 MB
3. Peak	**713.1 KB**	**5.23 MB**
4. After `zero_grad()`	416.7 KB	485.8 KB

Watch out

Measured well above prediction? Something is **alive** that you did not count — find it with the profiler.

Key takeaways

- A tensor is **metadata over storage**; views change strides, copies move bytes.
- The **last-dimension stride** decides coalescing; pass strides into your kernels.
- The forward pass is a few **ATen ops**: views are free, each kernel moves bytes; ours are memory-bound.
- Autograd **records** the forward and **replays it in reverse**: $\approx 2\times$ forward FLOPs; it keeps **activations** until backward.
- Every eager op pays **Python + dispatch + launch** on the CPU: small models are overhead-bound.
- Peak memory $\approx \theta$ + **optimizer state + activations + transient gradients** — predict it, then measure.

Next time, and readings

When	What
Lecture 2 (this week)	Graphs and transformations: torch.compile, torch.func.vmap
Lab 3	Trace the MLP step: memory and per-op time
T1 checkpoint	Due before spring break
Week 10	Framework and GPU interaction

READ BEFORE LECTURE 2

PyTorch docs: *Tensor Views*, *Understanding CUDA memory usage*, `torch.no_grad`.

PyTorch docs: *Introduction to torch.compile* — preview for Lecture 2.